

Redis + FastAPI Caching

Complete lesson notes: Redis, PostgreSQL, cache-aside, TTL, cache invalidation, and practical implementation

1. Big Picture

Caching means keeping a temporary copy of data in a fast storage system so that frequently requested data can be returned more quickly. In this lesson, Redis acts as the cache and PostgreSQL remains the permanent source of truth.

Typical architecture:

```
Client → FastAPI → Redis → PostgreSQL (only when needed)
```

Redis is normally much faster for simple key/value lookups than repeatedly querying a relational database. The goal is not to replace PostgreSQL. The goal is to reduce unnecessary database work and improve response time.

2. Source of Truth vs Cache

PostgreSQL: permanent application data. It is the source of truth.

Redis: temporary, fast copy of data that can usually be rebuilt from PostgreSQL.

If Redis is restarted and cached data disappears, the application should still work: a cache miss simply causes the application to read the data from PostgreSQL and put it back into Redis.

3. Why Cache Database Queries?

Imagine a product endpoint receives thousands of requests for the same product. Without caching, every request can cause a PostgreSQL query. With caching, the first request can populate Redis and subsequent requests can be served directly from Redis until the cache expires or is invalidated.

Without cache

```
Request → FastAPI → PostgreSQL → Response
```

With cache

```
Request → FastAPI → Redis → Response
                        ↘ PostgreSQL only on cache miss
```

4. Cache-Aside / Lazy Caching

The main pattern used in this lesson is called **cache-aside** (also commonly called lazy caching). The application explicitly checks Redis first. If the value is missing, it loads the data from PostgreSQL and then stores a copy in Redis.

```
Request
↓
Check Redis
├─ HIT → return cached data
└─ MISS → query PostgreSQL → store in Redis → return data
```

5. Connecting FastAPI to Redis

A reusable Redis client should be created once and reused by the application instead of constructing a new client inside every route.

```
import redis

redis_client = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)

def get_redis():
    return redis_client
```

decode_responses=True makes Redis return strings instead of raw bytes for normal string operations.

6. FastAPI Dependency Injection

FastAPI's Depends() lets a route receive a shared Redis client without manually creating or passing it every time.

```
from fastapi import Depends

@app.get("/example")
def example(redis_client=Depends(get_redis)):
    value = redis_client.get("some-key")
    return {"value": value}
```

FastAPI calls get_redis(), receives the Redis client, and injects it into the route parameter.

7. Redis GET and SET

The simplest cache operation is a key/value lookup.

```
redis_client.set("name", "Pujan")
value = redis_client.get("name")
```

For cached database records, a useful key naming convention is **product:{product_id}**. For example, product ID 1 becomes product:1.

8. JSON Serialization

Redis strings are convenient for storing serialized application data. A Python dictionary cannot simply be stored as a normal Redis string, so we convert it to JSON.

```
Python dict → json.dumps() → JSON string → Redis
Redis JSON string → json.loads() → Python dict

import json

product = {
    "id": 1,
    "name": "Mechanical Keyboard",
    "price": 120
}

redis_client.set("product:1", json.dumps(product), ex=300)
```

```
cached = redis_client.get("product:1")
product = json.loads(cached)
```

9. TTL and Expiration

TTL means **time to live**. It defines how long a Redis key should remain before Redis automatically removes it.

```
redis_client.set("product:1", json.dumps(product), ex=300)
ttl = redis_client.ttl("product:1")
```

TTL result	Meaning
Positive number	Number of seconds remaining
-1	Key exists but has no expiration
-2	Key does not exist

Important: if a key is set again with a new EX value, the expiration is reset based on that new SET. It does not add the new duration to the old remaining TTL.

10. Complete Cache-Aside GET

The following is the core implementation used in the lesson. It checks Redis first, falls back to PostgreSQL, and then caches the database result.

```
@app.get("/products/{product_id}")
def get_product(product_id: int):
    cached_result = redis_client.get(f"product:{product_id}")

    if cached_result:
        return {
            "data": json.loads(cached_result),
            "source": "Cached"
        }

    with engine.connect() as connection:
        result = connection.execute(
            text(
                "SELECT id, name, price "
                "FROM products WHERE id = :product_id"
            ),
            {"product_id": product_id},
        )

        row = result.fetchone()

        if row is None:
            return {"message": "Product not found"}

        product = {
            "id": row.id,
            "name": row.name,
            "price": row.price
        }

        redis_client.set(
            f"product:{product['id']}",
            json.dumps(product),
            ex=60 * 5
        )
```

```

return {
    "data": product,
    "source": "Database"
}

```

11. Understanding a Cache HIT

If Redis contains product:1, the application does not need to query PostgreSQL. It reads the JSON string, converts it back into a Python dictionary, and returns it.

```
GET /products/1 → Redis has product:1 → JSON decode → response with source=Cached
```

12. Understanding a Cache MISS

If product:1 is absent or has expired, Redis returns no value. The application then queries PostgreSQL, creates a dictionary, stores it in Redis, and returns the database result.

```
GET /products/1 → Redis MISS → PostgreSQL → Redis SET (5 min) → response with source=Database
```

13. Cache Invalidation

Caching creates a new problem: stale data. Suppose PostgreSQL says a product costs 150, but Redis still contains 120. A GET request that hits Redis would return the old value.

The common solution for this pattern is to invalidate the affected cache entry when the permanent data changes.

```

UPDATE PostgreSQL
↓
COMMIT
↓
DELETE corresponding Redis key
↓
Next GET → cache miss → PostgreSQL → fresh Redis value

```

14. Dynamic Update Endpoint

The update endpoint accepts a new price through a Pydantic request model. After updating PostgreSQL, it deletes the matching Redis key.

```

from pydantic import BaseModel

class ProductUpdate(BaseModel):
    price: int

@app.put("/products/{product_id}")
def update_product(product_id: int, product: ProductUpdate):
    with engine.connect() as connection:
        result = connection.execute(
            text("SELECT id FROM products WHERE id = :product_id"),
            {"product_id": product_id},
        )

        row = result.fetchone()

        if row is None:
            return {"message": "Product not found"}

        connection.execute(
            text(

```

```

        "UPDATE products "
        "SET price = :price "
        "WHERE id = :product_id"
    ),
    {
        "price": product.price,
        "product_id": product_id,
    },
)

connection.commit()

redis_client.delete(f"product:{product_id}")

return {
    "message": f"Product {product_id} updated successfully",
    "new_price": product.price,
}

```

15. Why Delete the Cache After COMMIT?

The database update should succeed before the cache is invalidated. Once PostgreSQL has committed the new value, deleting the Redis entry guarantees that the next GET cannot keep serving the old cached copy.

Important ordering: Update → COMMIT → DELETE cache. This lesson uses this simple cache-aside strategy; more complex distributed systems can require stronger consistency techniques.

16. Parameterized SQL

When using SQLAlchemy Core with raw SQL, dynamic values should be passed as parameters rather than inserted into an SQL string with Python f-strings.

```

text(
    "SELECT id, name, price "
    "FROM products WHERE id = :product_id"
)

connection.execute(
    statement,
    {"product_id": product_id}
)

```

This is safer and cleaner than building SQL by concatenating user-controlled values into the SQL string.

17. Docker Networking Reminder

When FastAPI runs directly on your host machine, localhost:6379 refers to the Redis container's published port. PostgreSQL was published as localhost:5433 because the container listens on 5432 internally.

If FastAPI itself later runs inside Docker Compose, use service names instead: Redis becomes redis:6379 and PostgreSQL becomes postgres:5432. Inside a container, localhost means that same container, not another Compose service.

18. Full Mental Model

The most important thing to remember is that Redis is not the permanent database in this architecture. It is an acceleration layer in front of PostgreSQL.

```
READ FLOW
Client → FastAPI → Redis?
├─ HIT → return cached value
└─ MISS → PostgreSQL → Redis SET → return value

WRITE FLOW
Client → FastAPI → PostgreSQL UPDATE → COMMIT → Redis DELETE
```

19. Example Request Sequence

A typical lifecycle for product 1 looks like this:

1. First GET: Redis miss → PostgreSQL returns price 120 → Redis stores price 120 for 5 minutes.
2. Second GET: Redis hit → returns price 120 without PostgreSQL.
3. PUT price=200: PostgreSQL changes to 200 → transaction commits → product:1 is deleted from Redis.
4. Next GET: Redis miss → PostgreSQL returns 200 → Redis stores 200.
5. Following GET: Redis hit → returns 200.

20. Common Mistakes

Mistake	Correct approach
Using a literal key like "product:{product_id}"	Use f"product:{product_id}"
Caching the whole database dictionary accidentally	Cache the requested product object
Forgetting json.dumps()	Serialize Python dict before storing
Forgetting json.loads()	Deserialize cached JSON before returning it
Updating DB but not invalidating Redis	Delete the corresponding cache key
Using f-strings for SQL values	Use SQL parameters such as :product_id
Creating a Redis client in every request	Reuse a shared client
Treating Redis as permanent storage	Keep PostgreSQL as source of truth

21. What You Should Be Able to Explain

After this lesson, you should be comfortable explaining: **What is a cache?** Why Redis is useful with PostgreSQL? What is cache-aside? What is a cache hit/miss? What does TTL mean? Why does a cache become stale? Why do we delete a Redis key after updating PostgreSQL? Why is PostgreSQL still the source of truth? And how do FastAPI, Redis, and PostgreSQL work together in the request flow?

22. Lesson Summary

The central idea is simple: **read from Redis first, fall back to PostgreSQL on a miss, cache the result, and invalidate the cached copy when the database changes.** Once this pattern is understood, many real-world Redis caching implementations become variations of the same idea.

23. Official Documentation & References

The following references are the main official documentation sources relevant to this lesson:

Redis Documentation

<https://redis.io/docs/latest/>

Redis SET command

<https://redis.io/docs/latest/commands/set/>

Redis EXPIRE command

<https://redis.io/docs/latest/commands/expire/>

redis-py Documentation

<https://redis.readthedocs.io/>

FastAPI Dependencies

<https://fastapi.tiangolo.com/tutorial/dependencies/>

SQLAlchemy Documentation

<https://docs.sqlalchemy.org/>

Pydantic Documentation

<https://docs.pydantic.dev/>

Note: These notes are a learning/reference document based on the Redis + FastAPI caching lesson. For current behavior, API details, and production guidance, always consult the official documentation above.